

ManufacturingDPP Contract Documentation

Digital Product Passport for Investment Casting Foundry (Aerospace)

Contract Name: ManufacturingDPP

Version: 1.0

Blockchain: IOTA EVM

Contract Overview

This is a single, reusable smart contract that creates and manages **Digital Product Passports (DPP)** as NFTs.

It tracks real physical aerospace parts (for example, turbine blades) from the moment they are cast in the foundry, through the entire supply chain, usage, maintenance, and finally to end-of-life recycling.

Important Note about Token IDs:

- Any token can be a "main equipment" (parent).
- There is **no strict rule** that only Token ID 0 can be the main equipment.
- Using Token ID 0 for the main turbine blade (and 1, 2, 3... for its parts) is just a **recommended convention** to keep things organised.
- You can have many main equipments in the same contract (e.g. Token 0 = Blade A, Token 100 = Blade B, etc.).

The contract uses a flexible parent-child system so you can link parts to any equipment you want.

1. Variables (State Variables, Mappings and Enums)

```
bytes32 public constant FOUNDRY_ROLE =  
keccak256("FOUNDRY_ROLE");  
bytes32 public constant MACHINING_ROLE =  
keccak256("MACHINING_ROLE");  
bytes32 public constant OEM_ROLE = keccak256("OEM_ROLE");  
bytes32 public constant MRO_ROLE = keccak256("MRO_ROLE");  
bytes32 public constant ADMIN_ROLE =  
keccak256("ADMIN_ROLE");
```

These are the five permission roles. Each role is assigned to specific wallet addresses.

```
mapping(uint256 => uint256) public parentOf;
```

Purpose: Links a part to its parent equipment. Any token can be a parent.

```
mapping(uint256 => uint256[]) public childrenOf;
```

Purpose: Shows all the parts that belong to one equipment. Works with any token ID.

```
mapping(uint256 => string) public tokenUris;
```

Purpose: Stores the IPFS link to the JSON file that contains all details about that token.

```
enum LifecycleStage { ManufacturingAndInspection,  
AssemblyAndIntegration, OperationAndUsage,  
MaintenanceAndService, EndOfLifeAndRecycling }
```

Purpose: The five official stages a part goes through during its life.

```
mapping(uint256 => LifecycleStage) public currentStage;
```

Purpose: Records the current stage of each token.

2. Events

```
event OwnershipTransferred( uint256 indexed tokenId, address  
from, address to, string newOwner, bytes32  
notarizationHash );
```

Purpose: Fired when ownership of any token changes.

```
event MaintenanceLogged( uint256 indexed tokenId, string  
details, bytes32 notarizationHash );
```

Purpose: Fired when maintenance or repair is recorded.

```
event CertificationAdded( uint256 indexed tokenId, string  
certType, string documentHash, bytes32 notarizationHash );
```

Purpose: Fired when a new certification is added.

```
event LifecycleStageUpdated( uint256 indexed tokenId,  
LifecycleStage newStage, bytes32 notarizationHash );
```

Purpose: Fired when the lifecycle stage changes.

```
event ProcessStepRecorded( uint256 indexed tokenId, string  
step, string details, bytes32 notarizationHash );
```

Purpose: Fired when a manufacturing process step is recorded.

```
event MetadataUpdated( uint256 indexed tokenId, string  
newURI, bytes32 notarizationHash );
```

Purpose: Fired whenever the IPFS metadata is updated.

3. Functions

MINTING FUNCTIONS

```
function mintEquipment(address to, uint256 equipmentId,  
string calldata initialURI) external onlyRole(ADMIN_ROLE)
```

Purpose: Creates the main equipment token (usually used for the complete turbine blade with token ID 0).

Who can call it: Only the wallet that has the ADMIN_ROLE (typically the

foundry owner or project admin).

Errors:

- "AccessControl: account is missing role" — if called by anyone without ADMIN_ROLE. **Parameters:**
- address to → Wallet address that will own the new token
- uint256 equipmentId → Token ID to mint (recommended value: 0)
- string calldata initialURI → IPFS link to the starting metadata JSON file

```
function mintPart(address to, uint256 partId, uint256  
equipmentId, string calldata initialURI) external  
onlyRole(ADMIN_ROLE)
```

Purpose: Creates a smaller part and automatically links it to the main equipment.

Who can call it: Only ADMIN_ROLE.

Errors:

- "AccessControl: account is missing role" — if called without ADMIN_ROLE
- "Equipment must exist" — if the equipmentId (parent) has not been minted yet **Parameters:**
- address to → Wallet address that will own the part
- uint256 partId → Token ID for the new part (should be 1 or higher)
- uint256 equipmentId → Parent equipment token ID
- string calldata initialURI → IPFS link to the part's metadata JSON

CORE LIFECYCLE FUNCTIONS

```
function updateMetadata(uint256 tokenId, string calldata  
newURI, bytes32 notarizationHash) external
```

Purpose: Updates the IPFS metadata JSON of any token. Used to add new process history, maintenance logs, certifications, etc.

Who can call it: Any wallet with FOUNDRY_ROLE, MACHINING_ROLE, OEM_ROLE, MRO_ROLE, or ADMIN_ROLE.

Errors:

- "Unauthorized" — if called by a wallet without any of the allowed roles
 - "Token does not exist" — if the tokenId has not been minted
- Parameters:**
- uint256 tokenId → Token to update
 - string calldata newURI → New IPFS link to the updated metadata JSON
 - bytes32 notarizationHash → IOTA notarization proof for this update

```
function recordOwnershipTransfer(uint256 tokenId, address  
to, string calldata newOwner, bytes32 notarizationHash)  
external
```

Purpose: Transfers ownership of the token to a new wallet address and records the new owner's name.

Who can call it: Any authorized role (FOUNDRY, MACHINING, OEM, MRO, or ADMIN).

Errors:

- "Unauthorized" — if called by a wallet without any allowed role
- "Token does not exist" — if the tokenId has not been minted

Parameters:

- uint256 tokenId → Token being transferred
- address to → New owner's wallet address
- string calldata newOwner → Human-readable name or company of the new owner
- bytes32 notarizationHash → IOTA notarization proof

```
function recordMaintenance(uint256 tokenId, string calldata details, bytes32 notarizationHash) external onlyRole(MRO_ROLE)
```

Purpose: Records maintenance or repair work performed on the part.

Who can call it: Only wallets with MRO_ROLE.

Errors:

- "AccessControl: account is missing role" — if called by anyone without MRO_ROLE
- Parameters:**
- uint256 tokenId → Token ID
 - string calldata details → Description of the maintenance work
 - bytes32 notarizationHash → IOTA notarization proof

```
function addCertification(uint256 tokenId, string calldata certType, string calldata documentHash, bytes32 notarizationHash) external
```

Purpose: Adds a new certification document (NDT report, X-Ray, QA report, etc.) to the token.

Who can call it: Any authorized role (FOUNDRY, MACHINING, OEM, MRO, or ADMIN).

Errors:

- "Unauthorized" — if called by an unauthorized wallet
- Parameters:**
- uint256 tokenId → Token ID
 - string calldata certType → Type of certification (e.g. "NDT", "X-Ray")
 - string calldata documentHash → IPFS hash of the actual document
 - bytes32 notarizationHash → IOTA notarization proof

```
function updateLifecycleStage(uint256 tokenId, LifecycleStage newStage, bytes32 notarizationHash) external
```

Purpose: Changes the current lifecycle stage of the token.

Who can call it: Any authorized role (FOUNDRY, MACHINING, OEM, MRO, or

ADMIN).

Errors:

- "Unauthorized" — if called by an unauthorized wallet
- "Token does not exist" — if the tokenId has not been minted

Parameters:

- uint256 tokenId → Token ID
- LifecycleStage newStage → New stage (from the enum)
- bytes32 notarizationHash → IOTA notarization proof

```
function recordProcessStep(uint256 tokenId, string calldata step, string calldata details, bytes32 notarizationHash) external onlyRole(FOUNDRY_ROLE)
```

Purpose: Records a specific manufacturing process step (only the foundry can do this).

Who can call it: Only wallets with FOUNDRY_ROLE.

Errors:

- "AccessControl: account is missing role" — if called by anyone without FOUNDRY_ROLE

Parameters:

- uint256 tokenId → Token ID
- string calldata step → Name of the process step (e.g. "Wax Pattern Creation")
- string calldata details → Parameters and results of the step
- bytes32 notarizationHash → IOTA notarization proof

ERC721 OVERRIDES

```
function tokenURI(uint256 tokenId) public view override returns (string memory)
```

Purpose: Returns the current IPFS URI where the full metadata JSON of the token is stored. This is the standard function used by NFT marketplaces and apps.

Who can call it: Anyone (view function, no gas required).

Errors: None (it will return an empty string if the token does not exist).

```
function supportsInterface(bytes4 interfaceId) public view override returns (bool)
```

Purpose: Tells other contracts and applications that this contract supports ERC-721 (NFT standard) and AccessControl.

Who can call it: Anyone (view function).

Errors: None.